

UNIT-I

Introduction to Scientific Programming and Python Basics

Short Answer Type Questions

1. What is scientific computing?

Scientific computing involves using computers to solve scientific problems through numerical analysis, simulations, and data processing. It is used in various fields like physics, biology, engineering, and finance. The key aspects of scientific computing **include:**

Numerical Computing, Data Analysis, Simulation and Modeling, Optimization.

2. Define Python interpreter?

Interpreters are the computer program that will convert the source code or an high level language into intermediate code (machine level language). It is also called **translator** in programming terminology. Interpreters executes each statements one by one. This process is called Interpretation. **For example Python is an interpreted language, PHP, Ruby, and JavaScript.**

3. Mention any two key features of Python.?

Python is a high-level, interpreted programming language that is beginner-friendly.

- It has simple and readable syntax.
- It supports multiple programming paradigms (procedural, object-oriented, functional).
- It has a large collection of libraries for scientific computing.

4. List any three numeric data types in Python.?

Three numeric data types in Python are:

int – Integer type (e.g., 5, -100, 0):Used to represent whole numbers without a decimal point.

float – Floating-point type (e.g., 3.14, -0.001, 2.0):Used to represent numbers with decimal points.

complex – Complex number type (e.g., 2 + 3j, -1j):Used to represent complex numbers with real and imaginary parts.

5. What is indentation in Python?

Python Indentation refers to the use of white-space (spaces or tabs) at the beginning of code line. It is used to define the code blocks. Indentation is crucial in Python because, unlike many other programming languages that use braces "{}" to define blocks, Python uses indentation.

Example:

if 10 > 5:

 print("This is true!")

 print("I am tab indentation")

print("I have no indentation")

6. Differentiate between global and local variables?

Feature	Global Variable	Local Variable
Scope	Accessible throughout the entire program	Accessible only within the function where it is declared
Declaration	Defined outside all functions	Defined inside a function
Modification	Can be modified inside functions using the global keyword	Modified directly inside its function only
Lifetime	Exists as long as the program runs	Exists only during the function execution

```
x=10 # Global variable. lifetime is as long as the program runs.
```

```
def my_function():
```

```
    x=5 # local variable.lifetime is limited to the function only
```

```
    print("Inside Function:",x)
```

```
my_function()
```

```
print("Outside Function:",x)
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
Inside Function: 5
```

```
Outside Function: 10
```

7. Explain the use of range()in Python with an example.

Range()

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax

range(start, stop, step)

Parameter Values

Parameter	Description
start	Optional. An integer specifying the starting position. Default is 0.
stop	Required. An integer specifying the stopping position (not included).
step	Optional. An integer specifying the increment step. Default is 1.

```
range (start, stop, step)
```

```
for i in range(0,10,2):
```

```
    print(i, end=" ")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
0 2 4 6 8
```

8. What are default arguments in Python functions?

Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument.

```
def person(name,age=17):          #here default age is 17
    print(f'Name: {name},Age: {age}')
person("John")    #if you don't provide the age ,then it takes the default age(17)
person("Mike",25) # here we have specified the age so default age will not be taken
person("harry")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
Name:John,Age:17
```

```
Name:Mike,Age:25
```

```
Name:harry,Age:17
```

9. Describe the role of comments in Python programming.

Comments in Python

Comments in Python are statements written within the code. They are meant to explain, clarify, or give context about specific parts of the code. The role of comments is to explain the working of a code, they have no impact on the execution or outcome of a program.

Example:

```
# This is a single-line comment
```

```
x = 5 # Assign 5 to x
```

10. What is the purpose of the import statement in Python?

- We can import the **functions, and classes** defined in a module to another module using the import statement.
- When the interpreter encounters an import statement, it imports the module if the module is present in the search path.
- Import statement allows access to standard libraries like **math, random, datetime, etc.**
- Import statement enables use of third-party libraries like **pandas, numpy, matplotlib.**

LONG ANSWER TYPE QUESTIONS

1. Write a Python program using if-elif-else to classify a number as positive, negative, or zero. Explain the output.

```
### 1. **Check if a Number is Positive, Negative, or Zero**
```

```
num = float(input("Enter a number: "))
```

```
if num > 0:
```

```
    print("The number is positive.")
```

```
elif num < 0:
```

```
    print("The number is negative.")
```

```
else:
```

```
    print("The number is zero.")
```

OUTPUT:(show all the possible outputs as it is for 7 marks)

CASE-1:

Enter a number: 25

The number is Positive.

CASE-2

Enter a number: -8

The number is Negative.

CASE-3

Enter a number: 0

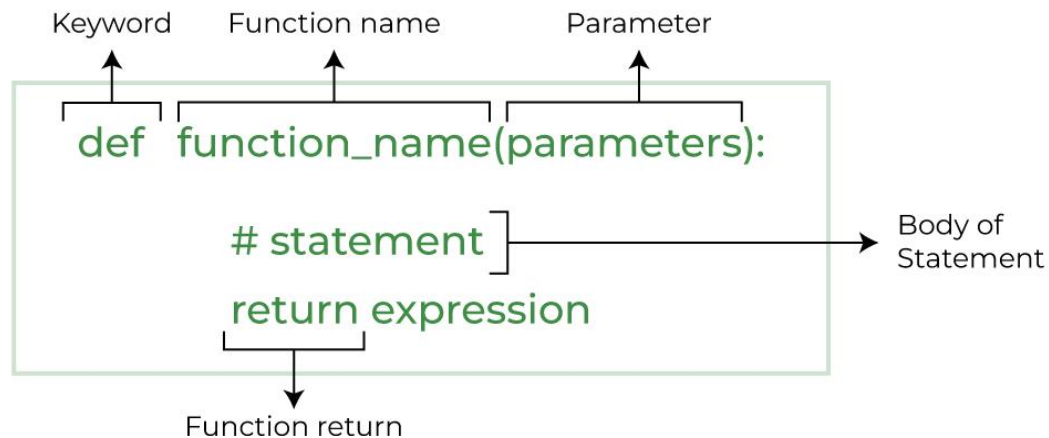
The number is Zero.

The user is asked to enter a number (it can be positive, negative, or zero).

The program checks:

- If the number is greater than 0 → prints "**Positive**"
- Else if it's less than 0 → prints "**Negative**"
- Else (i.e., it is exactly 0) → prints "**Zero**"

2. Demonstrate how to use a user-defined function with parameters and return type in Python.



Defining a User-Defined Function:

In Python, a user defined function is a reusable block of code. A user defined function is a block of code that can run when it is called.

A user defined function in Python is defined using the **def** keyword, followed by the **function name**, a parenthesized **list of parameters** (if any), and **a colon**. The **function body**, containing the code to be executed, must be indented. An optional **docstring** can be included as the first statement within the function body to describe its purpose. The **return statement**, if present, specifies the value(s) the function will output. Functions help organize code, making it more readable and easier to maintain. **They can take inputs (arguments), process them, and return outputs.**

HOW TO USE:

- Define the function using **def**
- Give it **parameters** (inputs)
- Do **some work** inside the function
- Use **return** to send a result back
- **Call** the function and give it values (arguments)
- **Store** the result and use it.

PROBLEM: Define a function named `my_adder` to take in 3 numbers and sum them.

```
def my_adder(a, b, c):  
    """  
    function to sum the 3 numbers  
    Input: 3 numbers a, b, c  
    Output: the sum of a, b, and c  
    author:Masrath parveen  
    date:23-04-2025  
    """  
    out = a + b + c          # this is the summation  
    return out  
#calling the function my_adder and store the returned value in d  
d = my_adder(1, 2, 3)  
print(d)  
# calling the same function but with different values  
d = my_adder(4, 5, 6)  
print(d)  
OUTPUT:  
PS C:\Users\masrath parveen\onedrive\desktop> python f.py  
6  
15
```

3. Compare and contrast list, tuple, and dictionary with examples. (This Question is FROM UNIT-2)

NOTE: **Compare** → Show how two or more things are **similar**

Contrast → Show how they are **different**

In Python, List, Tuple, Set, and Dictionary are built-in data structures used to store collections of items. Each has its own characteristics in terms of order, mutability, indexing, and data type.

Parameters	List	Tuple	Dictionary
Definition	A list is an ordered, mutable collection of elements.	A tuple is an ordered, immutable collection of elements.	A dictionary is an unordered collection of key-value pairs.
Syntax	Syntax includes square brackets [,] with ‘,’ separated data.	Syntax includes curved brackets (,) with ‘,’ separated data.	Syntax includes curly brackets { , } with ‘,’ separated key-value data.
Creation	A list can be created using the list() function or simple assignment to [].	Tuple can be created using the tuple() function.	A dictionary can be created using the dict() function.
Empty Data Structure	An empty list can be created by l = [].	An empty tuple can be created by t = ().	An empty dictionary can be created by {}.
Order	It is an ordered collection of data.	It is also an ordered collection of data.	Ordered collection in Python version 3.7, unordered in Python Version=3.6.
Duplicate Data	Duplicate data entry is allowed in a List.	Duplicate data entry is allowed in a Tuple.	Keys are unique, but two different keys CAN have the same value.
Indexing	Has integer based indexing that starts from ‘0’.	Also has integer based indexing that starts from ‘0’.	Has a Key based indexing i.e. keys identify the value.
Addition	New items can be added using the append() method.	Being immutable, new data cannot be added to it.	update() method updates specific key-value pair.
Deletion	Pop() method allows deleting an element.	Being immutable, no data can be popped/deleted.	pop(key) removes specified key along with its value.

Example program on list,tuple ,dictionary

```
a=[10,20,30] #LIST
print(a)
a.append(21)
a.extend([50, 60])
a.insert(2, 25)
print("After Additions:", a)
# Removing elements
a.remove(30)
popped = a.pop()
print("Popped Element:", popped)
a.pop(1)
del a[0]
print("After Removals:", my_list)
# Updating elements
my_list[1] = 99
print("After Update:", my_list)
TUPLES:
t1=(10,20,30,40,50)
t1[2]=95#not possible because tuples are immutable
print(t1)
# create dictionary using { } #DICTIONARY
d1 = {"name": 'john', "Rollno": 1234, "marks": 98.5}
print(d1)
# create dictionary using dict() constructor
d2 = dict(a = "apple", b = "banana", c = "mango")
print(d2)
# Access using key
print(d["name"])
# Access using get(): pass key as an argument
print(d.get("rollno"))
```

Conclusion:

- Use List when data needs to change and order matters.
- Use Tuple when the data must remain constant.
- Use Set to store unique values with no duplicates.
- Use Dictionary to associate keys with values for fast lookup.

4. Analyze how control structures affect the flow of a Python program. Give examples using loops and conditionals.

CONTROL STRUCTURES:

In Python, control structures dictate the order in which code is executed, enabling decision-making and repetition. The three primary types of control structures are: **sequential, selection, and repetition**. Sequential control structures execute statements in the order they appear. Selection structures, like if-else statements, allow code execution to branch based on conditions. Repetition structures, or loops (for and while), enable code blocks to be executed multiple times.

TYPES OF CONTROL STRUCTURES

1. Sequential

2. Selection

- a) If
- b) If-else
- c) If-elif-else
- d) Nested if-else
- e) Match cases.

3. Repetition./Iteration

- a) For loop
- b) While loop
- c) Nested loops(for and while)

4. Loop Control Statements:

- a) Break
- b) Continue
- c) Pass

1. Sequential Control Structures:

- These structures execute statements in the order they are written in the code.
- For example, if you have three lines of code, they will be executed one after the other, from top to bottom.

2. Selection Control Structures (Conditional Statements):

- These structures allow the program to make decisions based on specific conditions and execute different code blocks accordingly.
- The most common selection structure is the if-else statement.
- **if statement:** Checks if a condition is true, and if it is, it executes the code block associated with it.
- **else statement:** Executes the code block associated with it if the if condition is false.
- **elif statement:** (short for "else if") allows you to check multiple conditions and execute different code blocks based on the first true condition.

Programs On Selection Control Structures

```
#### 1. **Check if a Number is Positive, Negative, or Zero**  
num = float(input("Enter a number: "))  
if num > 0:  
    print("The number is positive.")  
elif num < 0:  
    print("The number is negative.")  
else:  
    print("The number is zero.")
```

Repetition Control Structures (Loops):

These structures allow the program to execute a block of code repeatedly until a certain condition is met. Python has two primary types of loops: for loops and while loops.

A **while loop** is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

A **for loop** in Python is used to iterate over a sequence (such as a list, tuple, string, or range) or other iterable objects. It executes a block of code for each item in the sequence.

```
### **1. Print Numbers from 1 to 10 (Using `for` Loop)**  
for i in range(1, 11):  
    print(i)  
  
### **2. Print Even Numbers from 1 to 20 (Using `while` Loop)**  
i = 2  
while i <= 20:  
    print(i)  
    i += 2
```

Loop Control Statements:

- These statements can be used within loops to control their execution.
- **break**: Terminates the loop prematurely, exiting it.
- **continue**: Skips the current iteration of the loop and proceeds to the next one.
- **pass**: Acts as a placeholder, meaning it does nothing but is necessary to satisfy the syntax when a code block is required but no action needs to be taken.

5. Evaluate the advantages and limitations of using Python for scientific computing.

Python for Scientific Computing: Advantages and Limitations

Python is widely used in scientific computing due to its simple syntax and powerful libraries. However, it can be slower for heavy computations.

ADVANTAGES:

Easy to Learn: Python is beginner-friendly with readable syntax, allowing scientists to focus on logic rather than complex code.

Powerful Libraries:

NumPy – fast numerical calculations

SciPy – scientific functions like integration and optimization

Pandas – data analysis and manipulation

Matplotlib / Seaborn – data visualization

Scikit-learn – machine learning

TensorFlow / PyTorch – deep learning

Strong Community Support: Lots of tutorials, forums, and free resources available.

Integration with Other Languages: Python can use C/C++ for faster performance.

Multi-purpose: Can be used for data analysis, automation, web apps, and more.

LIMITATIONS:

Slower Speed: Python is slower than C or Java for CPU-heavy tasks.

GIL (Global Interpreter Lock): Limits multi-threading; not ideal for parallel tasks.

High Memory Usage: Python uses more memory, which can be a problem for big data.

Dynamic Typing: Errors may show up at runtime instead of during coding.

Dependency Management Issues: Installing scientific libraries can sometimes lead to version conflicts, especially across platforms or older systems.

6. Discuss the significance of using libraries such as NumPy, Matplotlib, and SciPy in scientific programming.

NumPy, SciPy, and Matplotlib are essential libraries that transform Python into a powerful tool for scientific computing.

NumPy (Numerical Python)

NumPy provides the fundamental **N-dimensional array** for fast numerical operations. It's significantly quicker than standard Python lists because it uses optimized C code. Many other scientific libraries are built on top of NumPy.

SciPy (Scientific Python)

SciPy offers a wide range of **advanced scientific algorithms**. It includes tools for numerical integration, optimization, signal processing, and statistics, saving researchers time by providing ready-to-use, reliable functions.

```
from scipy.linalg import solve
import numpy as np

# Coefficient matrix A
A = np.array([[2, 3],
              [3, 4]])

# Right-hand side vector b
b = np.array([8, 11])

# Solve the system Ax = b
x = solve(A, b)

# Print the solution
print("Solution:")
print("x =", x[0])
print("y =", x[1])
```

`scipy.linalg.solve()` solves $Ax=b$ where:

A is the coefficient matrix

b is the constants vector

It returns the values of x and y that satisfy both equations.

Numpy provides the ndarray

Matplotlib (Mathematical Plotting Library)

Matplotlib is crucial for **data visualization**. It allows scientists to create various plots (line, scatter, histograms, etc.) to understand data, identify patterns, and produce high-quality figures for publications and presentations.

```
import matplotlib.pyplot as plt
# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 8, 7]
# Create a line plot
plt.plot(x, y)
# Add labels and title
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Basic Line Plot")
# Show the plot
plt.show()
```

In short, NumPy handles fast data structures, SciPy provides the complex algorithms, and Matplotlib helps visualize the results. Together, they form the core of scientific computing in Python, making it an indispensable tool for research.

7. Design a Python module containing at least one function and one variable. Then, show how to import and use them in another script.

Create your own module.

- Every file acts as a module in python.

Step1: You create a module just by saving Python code in a file with a **.py** extension.

Step2: create another file. Now use the functions/variables etc from the first file into the current file using **syntax : import module_name**

Example:step1: creating own.py

```
# the name of this file is own.py
x=56
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
def mul(a,b):
    return a*b
def fact(n):
    if n == 0:
        return 1
    else:
        return n * fact(n - 1) # Recursive case
```

Step 2:creating calculate.py

```
# the name of this file is calculate.py
import own # here own.py acts as a module
print(own.x) # from own.py we are printing the value of x
print(own.add(5,2)) # from own.py we are using add function
print(own.fact(5))
OUTPUT: PS C:\Users\masrath parveen\PycharmProjects\div3> python calculate.py
56
7
120
```

You can use import from syntax as well.

```
from own import sub,mul
print(sub(9,2))
print(mul(2,5))
OUTPUT:
PS C:\Users\masrath parveen\PycharmProjects\div3> python calculate.py
7
10
```


8. Create a recursive function in Python to compute the factorial of a number and explain the flow of execution.

RESURSION/RECURSIVE FUNCTIONS:

A recursive function is a function that calls itself to solve a problem. It has two key parts.

1. **Base Case:** A condition to end the recursion and stop the function from calling itself indefinitely
2. **Recursive Case:** the part of the function where it calls itself to break the problem into smaller sub problems.

Example: factorial of a number flow of execution.

factorial(5) # to slove fact(5) we need 5* fact(4)
= 5 * factorial(4) # to slove fact(4) we need 4* fact(3)
= 5 * 4 * factorial(3) # and so on
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 * factorial(0) # now we know fact(0) is 1. so we do
backtracking to solve the factorial of 5
= 5 * 4 * 3 * 2 * 1 * 1 = 120

```
#Program to find the factorial of a number
def fact(n):
    if n == 0:          # Base case
        return 1
    else:
        return n * fact(n - 1) # Recursive case

print(fact(5)) # Output: 120
```

9. Write a program in MATLAB to calculate and plot the sine function. Explain each command used.

Plotting mathematical functions is a common task in scientific computing, and MATLAB makes it particularly easy with its built-in mathematical functions and powerful plotting capabilities.

```
x = 0:0.1:10;           % X values from 0 to 10 with step  
of 0.1  
y = sin(x);             % Y values as sine of x  
  
plot(x, y);             % Basic plot  
title('Sine Wave');  
xlabel('x-axis');  
ylabel('y = sin(x)');  
grid on;                % Adds grid lines
```

x = 0:0.1:10; Creates a vector x from 0 to 10 with step size 0.1 (high resolution).

y = sin(x); Calculates the sine of each x value and stores the result in y.

plot(x, y); Plots the sine wave using x and y values as a 2D line graph.

title('Sine Wave'); Adds the title "Sine Wave" at the top of the graph.

xlabel('x-axis'); Labels the horizontal axis as "x-axis".

ylabel('y = sin(x)'); Labels the vertical axis to show it represents sine values.

grid on; Adds grid lines to make the graph easier to read and interpret.

10. Design a solution using nested loops in Python to display a multiplication table up to 10.

```
for i in range(1, 11):  
    for j in range(1, 11):  
        print(f"{i * j:4}", end=" ")  
    print()
```

EXPLANATION:

- The program **generates and displays the multiplication table from 1 to 10.**

It prints each multiplication in the format **$i \times j = \text{result}$** .

- **for i in range(1, 11):** This is the outer loop, which iterates from 1 to 10 (inclusive). Each value of i represents a row number (or the multiplicand).
- **for j in range(1, 11):** This is the inner loop, which also runs from 1 to 10. Each value of j represents a multiplier. For every i, the inner loop runs 10 times to complete that row of the multiplication table.
- **product = i * j :** The actual multiplication is performed here. The result of multiplying the current values of i and j is stored in the variable product.
- **print(f"{i} x {j} = {product}", end="\n"):** This line uses **formatted strings (f-strings)** to display the multiplication in a readable form like: **$2 \times 3 = 6$**
- The **end="\n"**: is optional here as it's the default behavior; it means each multiplication is printed on a new line.
- **print():** This empty **print()** is used after each row to create a blank line for better readability and separation between tables of different numbers (i.e., after each i).
- **print("\nMultiplication table generated successfully!"):** After both loops finish, this line prints a confirmation message. **\n** adds a new line before the message to separate it from the table above.

Final Output Example (Partial):

```
1 x 1 = 1
1 x 2 = 2
...
1 x 10 = 10

2 x 1 = 2
2 x 2 = 4
...
2 x 10 = 20

...

10 x 10 = 100
```